

# **Sistema para taller mecánico**

## **Identificación del proyecto:**

- Nombre: Taller Manager
- Versión:
- Autor:
- Stack: Node.js (Express), React, MySQL.

## **Visión general y objetivos:**

Centralizar la gestión de un taller mecánico mediante una plataforma web que permita administrar clientes, vehículos, turnos, reparaciones y facturación de forma rápida y organizada.

## **Los objetivos de este sistema son:**

- Registrar clientes y vehículos.
- Gestionar turnos de reparación.
- Controlar el estado de cada trabajo.
- Generar presupuestos y facturas.
- Visualizar historial de reparaciones por vehículo.

## **Requerimientos Funcionales:**

- Registrar clientes con datos personales y de contacto.
- Permitir agregar más de un vehículo a un cliente.
- Registrar turnos para revisiones o reparaciones.
- Actualizar el estado de una reparación (Pendiente, En proceso, Finalizada).
- Almacenar el historial de reparaciones de cada vehículo.
- Generar presupuestos y facturas.
- Permitir búsquedas por patente, cliente o vehículo.
- Aviso de alerta de turnos programados.

## **Requerimientos no funcionales:**

- Interfaz estética, con responsive para más de un dispositivo.
- Modo oscuro.
- Funciones de pago online.

## Arquitectura de Datos (Modelado en MySQL)

### Tabla clientes:

| Campo     | Tipo    |                |
|-----------|---------|----------------|
| Id        | INT     | AUTO-INCREMENT |
| Nombre    | VARCHAR | NOT NULL       |
| Teléfono  | INT     | NOT NULL       |
| Dirección | VARCHAR | NULL           |

### Tabla vehículos:

| Campo      | Tipo    |          |
|------------|---------|----------|
| Patente    | VARCHAR | PK       |
| Id cliente | INT     | FK       |
| Marca      | VARCHAR | NOT NULL |
| Año        | INT     | NULL     |
| Modelo     | VARCHAR | NOT NULL |
| Km         | INT     | NULL     |

### Diseño de la API

#### Clientes:

GET / → Obtener todos los clientes.

GET /:id → Obtener cliente específico.

POST / → Crear cliente.

PUT /:id → Actualizar cliente.

DELETE /:id → Eliminar cliente.

#### Vehículos:

GET / → Obtener todos los vehículos.

GET /:id → Obtener vehículo específico.

POST / → Registrar vehículo.

PUT /:id → Actualizar vehículo.

DELETE /:id → Eliminar vehículo.

## **Flujo de datos:**

**Request:** El usuario solicita información desde el frontend.

**API:** Express recibe la petición y valida autenticación.

**Database:** Sequelize consulta MySQL.

**Process:** El backend procesa y organiza los datos.

**Response:** Se devuelve un JSON optimizado.

**Render:** React actualiza la interfaz dinámicamente.

## **Tecnologías Utilizadas**

**Backend:** Node.js, Express, MySQL.

**Frontend:** React, CSS, Axios y React.

## **Herramientas adicionales**

Postman, Git y GitHub, Figma.